Wagonblast, Gregory
INFSCI 2440

Assignment 4 Report: Natural Language Understanding

Introduction

In order to begin this assignment, I used the following tutorial provided in the assignment documentation → How to setup and use Stanford CoreNLP Server with Python - Khalid Alnajjar. I downloaded stanford-corenlp-4.5.10, unzipped, and ran the following command to start the CoreNLP Server → "java -mx4g -cp "*" edu.stanford.nlp.pipeline.StanfordCoreNLPServer -annotators "tokenize,ssplit,pos,lemma,parse,sentiment" -port 9000 -timeout 30000". After the CoreNLP Server is launched, we can see it listening through the command line.

```
[main] INFO CoreNLP - --- StanfordCoreNLPServer#main() called ---
[main] INFO CoreNLP - Server default properties:
                        (Note: unspecified annotator properties are English defaults)
                        annotators = tokenize,ssplit,pos,lemma,parse,sentiment
                        inputFormat = text
                        outputFormat = json
                        prettyPrint = false
[main] INFO CoreNLP - Threads: 12
[main] INFO CoreNLP - Starting server...
[main] INFO CoreNLP - StanfordCoreNLPServer listening at /[0:0:0:0:0:0:0:0]:9000
[pool-1-thread-2] INFO CoreNLP - [/[0:0:0:0:0:0:0:1]:63958] API call w/annotators tokenize,pos,lemma,ner,parse,depparse,
dcoref,relation
```

I am utilizing the StandfordNLP class in the linked website to communicate with the CoreNLP Server through the use of a Python client. This client uses the Python package "standfordcorenlp". Now, the extraction task is possible.

Task 1: Extraction

First off, the extraction module relies on the dependency parser from Standford CoreNLP in order to identify semantic relationships between words. This specifically looks for links between attributes (nouns) and values (adjectives). I used the generated annotation "EnhancedPlusPlusDependenciesAnnotation" in order to provide the most semantically rich set of dependencies.

Second, when mapping relationships I looked for specific dependency tags like amod (adjectival modifier) and nsubj (nominal subject). For example, "great service" is an amod relationship while "service was great" is a nsubj relationship. The first has the **governor noun** as the **attribute** and the **dependent adjective** as the **value**, the second has the **dependent noun** as the **attribute** and the **governor adjective** as the **value**. We should use this approach because it works as a powerful filter by taking advantage of the rules of English. Additionally, this helped to provide a standardized output by structuring the extracted opinion as [attribute, value].

Lastly, in order to ensure the quality of the extraction I used part-of-speech (POS) tags in order to confirm that the noun of the extracted sentence was indeed the attribute and the adjective the value. If this simple check passed the test, I used lemmatization to group similar words (e.g. "pricing", "price", "prices"). Finally, I ensured that an extracted opinion could contain multiple review ids, but ensured that there were no duplicate review ids.

Task 2: Similarity

For this task, the similarity is measured using Word2Vec embeddings, which represent words as multi-dimensional vectors → Google Code Archive - Long-term storage for Google Code Project Hosting.. The Word2Vec object that is instantiated in the code uses cosine similarity to calculate the similarity between two words. Fundamentally, cosine similarity is calculated as the dot product of two vectors divided by the product of two vectors magnitude. Equation → $(A \cdot B) \div (\| A \| \cdot \| B \|)$. I simply check if the query attribute/value (noun/adjective) from our user "Lily" is similar to any of the extracted opinions attributes/values (nouns/adjectives). Then, from the Assignment4Main.py file I was able to tune the cosine similarity threshold. The output from tuning the threshold may be viewed in the output section of this report below.

Note: I added exception handling to my findSimiliarOpinions method in order to account for opinions not existing in the Word2Vec vocabulary.

Output

Extraction Output: The Task 1 output does not change and is consistent with the information provided in the assignment documentation.

[co-worker, old] appears in review [1]

[meal, wonderful] appears in review [1]

[salad, huge] appears in review [1]

[salad, delicious] appears in review [1]

[stick, little] appears in review [1]

[stick, delicious] appears in review [1]

[service, excellent] appears in review [1 , 2]

[place, great] appears in review [1]

[experience, upscale] appears in review [1]

[spot, other] appears in review [1]

Wagonblast, Gregory
INFSCI 2440

[restaurant, old] appears in review [2]

[woodwork, ornate] appears in review [2]

[tablecloth, white] appears in review [2]

[potato, red] appears in review [2]

[potato, skinned] appears in review [2]

[meat, tender] appears in review [2]

[meat, flavorful] appears in review [2]

[slaw, delicious] appears in review [2]

[atmosphere, nice] appears in review [3]

[list, local] appears in review [3]

[list, great] appears in review [3]

[potion, huge] appears in review [3]

[menu, huge] appears in review [3]

[thing, good] appears in review [3]

[note, positive] appears in review [4]

[meal, delicious] appears in review [4]

[fries, amazing] appears in review [4]

[ambiance, nice] appears in review [4]

[selection, great] appears in review [4]

[server, rude] appears in review [4]

[restaurant, small] appears in review [4 , 5]

[table, own] appears in review [4]

[ride, useless] appears in review [5]

[meal, good] appears in review [5]

[fan, huge] appears in review [5]

[service, great] appears in review [5 , 1 4]

[drink, large] appears in review [5]

[sandwich, french] appears in review [5]

[restaurant, nice] appears in review [5]

[atmosphere, great] appears in review [5]

[food, great] appears in review [5]

[rotation, regular] appears in review [6]

[food, excellent] appears in review [6 , 1 3]

[devonshire, awesome] appears in review [6]

[desert, recent] appears in review [6]

[desert, mini] appears in review [6]

[deal, great] appears in review [6]

[selection, good] appears in review [6]

[quality, bad] appears in review [7]

[day, bad] appears in review [7]

[food, good] appears in review [8]

[food, hearty] appears in review [8]

[price, decent] appears in review [8]

[service, warm] appears in review [8]

[fish, battered] appears in review [8]

[good, bad] appears in review [8]

[service, bad] appears in review [9]

[finger, homemade] appears in review [1 0]

[burger, good] appears in review [1 0]

[beer, good] appears in review [1 0]

[value, great] appears in review [1 0]

[menu, large] appears in review [1 1]

[portion, large] appears in review [1 1]

[price, reasonable] appears in review [1 1]

[quality, excellent] appears in review [1 1]

[menu, entire] appears in review [1 1]

[speciality, other] appears in review [1 1]

[favorite, top] appears in review [1 2]

[favorite, local] appears in review [1 2]

[feeling, warm] appears in review [1 2]

Wagonblast, Gregory
INFSCI 2440

[deal, cheap] appears in review [1 2]

[portion, generous] appears in review [1 2]

[order, particular] appears in review [1 2]

[course, main] appears in review [1 2]

[soup, french] appears in review [1 2]

[summer, last] appears in review [1 3]

[time, wonderful] appears in review [1 3]

[service, solid] appears in review [1 3]

[group, large] appears in review [1 3 , 1 6]

[grill, good] appears in review [1 4]

[turkey, roasted] appears in review [1 4]

[food, fresh] appears in review [1 4 , 1 8]

[food, interesting] appears in review [1 4]\

taco, good] appears in review [1 5]

[waiter, slow] appears in review [1 5]

[time, long] appears in review [1 5]

[thing, second] appears in review [1 5]

[thing, sure] appears in review [1 5]

[food, cold] appears in review [1 5]

[food, satisfying] appears in review [1 6]

[food, typical] appears in review [1 6]

[food, american] appears in review [1 6]

[way, bad] appears in review [1 6]

[option, vegan] appears in review [1 6]

[salad, large] appears in review [1 7]

[salad, greek] appears in review [1 7]

[nacho, good] appears in review [1 7]

[fries, great] appears in review [1 7]

[waiter, friendly] appears in review [1 7]

[waiter, attentive] appears in review [1 7]

[wine, decent] appears in review [1 8]

[food, fast] appears in review [1 8]

[restaurant, busy] appears in review [1 9]

[hope, high] appears in review [1 9]

[food, ok] appears in review [1 9]

[service, good] appears in review [2 0]

[ambience, pleasant] appears in review [2 0]

[pittsburgher, old] appears in review [2 0]

Similarity Output: Base case, cosine_sim = 0.8. The base case successfully identified similar opinions in the review. That said, the threshold is too high to identify any meaningful relationships for [atmosphere, good] and [food, delicious].

query opinion [service, good] has similar opinions:

[service, good] appears in review [2 0]

query opinion [service, bad] has similar opinions:

[service, bad] appears in review [9]

query opinion [atmosphere, good] has similar opinions:

query opinion [food, delicious] has similar opinions:

Similarity Output: Increasing the threshold, cosine_sim = 1.0. This is the same output as the base case, now I know why cosine_sim = 0.8 is the default threshold provided!

query opinion [service, good] has similar opinions:

[service, good] appears in review [2 0]

query opinion [service, bad] has similar opinions:

[service, bad] appears in review [9]

query opinion [atmosphere, good] has similar opinions:

query opinion [food, delicious] has similar opinions:

Wagonblast, Gregory
INFSCI 2440

Similarity Output: Decreasing the threshold, cosine_sim = 0.7. Here is where things start to get interesting. We mostly see success, including an additional finding between [atmosphere, good] and [atmosphere, great]. That said, we are seeing a case of antonym confusion, [service, good] and [service, bad] are being output as similar. This is a failure, although these words may be mathematically similar, they are not linguistically similar. Lastly, we still haven't found a similar opinion for [food, delicious].

query opinion [service, good] has similar opinions:

[service, great] appears in review [5 , 1 4]

[service, bad] appears in review [9]

[service, good] appears in review [2 0]

query opinion [service, bad] has similar opinions:

[service, bad] appears in review [9]

[service, good] appears in review [2 0]

query opinion [atmosphere, good] has similar opinions:

[atmosphere, great] appears in review [5]

query opinion [food, delicious] has similar opinions:

Similarity Output: Decreasing the threshold more, cosine_sim = 0.6. We are now seeing additional matches, which is a success. This success is our first case of identifying a synonym between atmosphere and ambiance… interesting. That said, we still have, and will continue to have antonym confusion when decreasing the threshold. I will continue decreasing the threshold until we get a match for [food, delicious]!

query opinion [service, good] has similar opinions:

[service, excellent] appears in review [1 , 2]

[service, great] appears in review [5 , 1 4]

[service, bad] appears in review [9]

[service, good] appears in review [2 0]

query opinion [service, bad] has similar opinions:

[service, bad] appears in review [9]

[service, good] appears in review [2 0]

query opinion [atmosphere, good] has similar opinions:

[atmosphere, nice] appears in review [3]

[ambiance, nice] appears in review [4]

[atmosphere, great] appears in review [5]

query opinion [food, delicious] has similar opinions:

Similarity Output: Decreasing the threshold more and more, cosine_sim = 0.5. We finally see similar opinions found for query [food, delicious]! We are seeing 12/14 results that make sense. Additionally, we are identifying another synonym between food and meal.

query opinion [service, good] has similar opinions:

[service, excellent] appears in review [1 , 2]

[service, great] appears in review [5 , 1 4]

[service, bad] appears in review [9]

[service, solid] appears in review [1 3]

[service, good] appears in review [2 0]

query opinion [service, bad] has similar opinions:

[service, bad] appears in review [9]

[service, good] appears in review [2 0]

query opinion [atmosphere, good] has similar opinions:

[atmosphere, nice] appears in review [3]

[ambiance, nice] appears in review [4]

[atmosphere, great] appears in review [5]

query opinion [food, delicious] has similar opinions:

[meal, wonderful] appears in review [1]

[meat, flavorful] appears in review [2]

[meal, delicious] appears in review [4]

[food, hearty] appears in review [8]

Similarity Output: Decreasing the threshold more and more and more, cosine_sim = 0.4. Mostly good output, these are the best results so far. This may be the optimal threshold for our particular testing. I will find a threshold that will be too lenient next.

query opinion [service, good] has similar opinions:

[service, excellent] appears in review [1 , 2]

[service, great] appears in review [5 , 1 4]

[service, bad] appears in review [9]

[service, solid] appears in review [1 3]

[service, good] appears in review [2 0]

query opinion [service, bad] has similar opinions:

[service, bad] appears in review [9]

[service, good] appears in review [2 0]

query opinion [atmosphere, good] has similar opinions:

[atmosphere, nice] appears in review [3]

[ambiance, nice] appears in review [4]

[atmosphere, great] appears in review [5]

[ambience, pleasant] appears in review [2 0]

query opinion [food, delicious] has similar opinions:

[meal, wonderful] appears in review [1]

[meat, flavorful] appears in review [2]

[meal, delicious] appears in review [4]

[food, hearty] appears in review [8]

[food, satisfying] appears in review [1 6]

Similarity Output: Decreasing the threshold more and more and more and more, cosine_sim = 0.2. The output begins to breakdown here by being too lenient with similar results. That said, It is successfully finding a mathematical similarity between food → salad → slaw → fries → taco → nachos. I think that this is useful information, but realistically, this system would be too lenient to put into production.

query opinion [food, delicious] has similar opinions:

[meal, wonderful] appears in review [1]

[salad, delicious] appears in review [1]

[meat, tender] appears in review [2]

[meat, flavorful] appears in review [2]

[slaw, delicious] appears in review [2]

[atmosphere, nice] appears in review [3]

[meal, delicious] appears in review [4]

[fries, amazing] appears in review [4]

[ambiance, nice] appears in review [4]

[meal, good] appears in review [5]

[restaurant, nice] appears in review [5]

[atmosphere, great] appears in review [5]

[food, great] appears in review [5]

[food, excellent] appears in review [6 , 1 3]

[food, good] appears in review [8]

[food, hearty] appears in review [8]

[burger, good] appears in review [1 0]

[beer, good] appears in review [1 0]

[turkey, roasted] appears in review [1 4]

[food, fresh] appears in review [1 4 , 1 8]

[food, interesting] appears in review [1 4]

[taco, good] appears in review [1 5]

[food, satisfying] appears in review [1 6]

[nacho, good] appears in review [1 7]

[fries, great] appears in review [1 7]

[waiter, friendly] appears in review [1 7]

[waiter, attentive] appears in review [1 7]

[wine, decent] appears in review [1 8]

[ambience, pleasant] appears in review [2 0]

Summary

The extraction step went well, the output is consistent with the information in the assignment documentation given. The similarity step went well too, although there is no perfect threshold! If I make the threshold too high, we lose a lot of results. If I make the threshold too low, we get too many incorrect matches for a production ready system. The threshold of 0.4 was as close as I got to optimal, this was my best results.

There was plenty of success in testing the threshold. This included matching many synonyms correctly, including identifying the relation ship between [ambiance, nice] and [atmosphere, good]. Additionally, I was able to track that [service, great] appeared in both reviews 5 and 14.

Also, I was able to match queries like “service” to opinions like “waiter” and “server”, which are contextually related.

Finally, there were plenty of failure cases. This included antonym confusion between mathematically similar words such as the cases of [service, good] and [service, bad]. Also, there was also a typo in review 8, “Haven’t had a bad good here yet.”. This resulted in an extraction that didn’t make sense → [good, bad]. Lastly, we saw that as the threshold decreased, we got back too many similar reviews that weren’t actually similar. For example, [food, delicious] and [waiter, friendly]. Although, there is some relationship between the waiter and the food, the waiter really has nothing to do with the quality of the food.

Example picture of processing when the CoreNLP Server is running.

```
[pool-1-thread-12] INFO CoreNLP - [/[0:0:0:0:0:0:0:1]:53350] API call w/annotators tokenize,pos,lemma,ner,parse,depparse
,dcoref,relation
Delicious old school bar/restaurant. Love the ornate woodwork and white tablecloths. Service was absolutely excellent. I
 had the pot roast with the red skinned mashed potatoes and cole slaw. The meat was so tender and flavorful, and the pot
atoes...yum! The Cole slaw was delicious too, and I'm VERY picky about Cole slaw.
[pool-1-thread-12] INFO edu.stanford.nlp.pipeline.StanfordCoreNLP - Adding annotator tokenize
[pool-1-thread-12] INFO edu.stanford.nlp.pipeline.StanfordCoreNLP - Adding annotator pos
[pool-1-thread-12] INFO edu.stanford.nlp.pipeline.StanfordCoreNLP - Adding annotator lemma
[pool-1-thread-12] INFO edu.stanford.nlp.pipeline.StanfordCoreNLP - Adding annotator ner
[pool-1-thread-12] INFO edu.stanford.nlp.pipeline.StanfordCoreNLP - Adding annotator parse
[pool-1-thread-12] INFO edu.stanford.nlp.pipeline.StanfordCoreNLP - Adding annotator depparse
[pool-1-thread-12] INFO edu.stanford.nlp.pipeline.StanfordCoreNLP - Adding annotator dcoref
[pool-1-thread-12] INFO edu.stanford.nlp.pipeline.StanfordCoreNLP - Adding annotator relation
[pool-1-thread-1] INFO CoreNLP - [/[0:0:0:0:0:0:0:1]:53351] API call w/annotators tokenize,pos,lemma,ner,parse,depparse,
dcoref,relation
```